

TOPS Technologies

Security Tools & CLI

Assessment A5 — Complete Answers

Theme: Cyber Security

Total Time: 3 Hours

Assessment 5

Section A — Concept Application

Question 1: Multi-factor verification of a password-reset email

Answer:

As a security analyst, I would use a multi-layered verification process to determine whether the password-reset email is legitimate or a credential-harvesting attempt.

Step 1: Verify the request through an independent channel

I would ask the employee whether they recently requested a password reset. I would not use the telephone number, email address, or link supplied in the suspicious message. Instead, I would contact the employee through a known corporate communication channel.

Step 2: Inspect the sender's identity

I would examine the sender's complete email address, display name, and domain. I would look for misspellings, extra characters, lookalike domains, and unexpected external senders impersonating the company's IT department.

Step 3: Examine the email headers

I would inspect the From, Sender, Reply-To, Return-Path, Received, and Authentication-Results fields. I would check for inconsistencies between the visible sender and the actual sending infrastructure.

Step 4: Verify SPF, DKIM, and DMARC

I would examine the email authentication results:

- SPF: Checks whether the sending IP address is authorized by the domain's SPF policy.
- DKIM: Verifies the cryptographic signature associated with the message.
- DMARC: Checks alignment between the visible From domain and SPF or DKIM authentication and applies the domain's published policy.

Passing these checks is useful evidence, but it does not automatically prove that the email is safe. A compromised legitimate account can also send phishing messages.

Step 5: Inspect the embedded link

I would examine the destination URL without opening it. I would check the registered domain, subdomains, HTTPS certificate information, URL redirects, and whether the destination resembles the company's genuine identity provider.

Step 6: Verify the reset through the official service

I would instruct the employee to open the company's known identity-provider portal using a trusted bookmark or manually entered address. If a password reset is required, the employee should initiate it from that portal rather than through the email.

Step 7: Check for additional warning signs

I would look for urgency, threats of account suspension, unexpected attachments, requests for MFA codes, and requests to approve an unfamiliar login.

Conclusion: I would classify the email using the combined evidence from the employee's confirmation, email headers, authentication results, URL analysis, and the official password-reset portal. If the evidence indicates phishing, I would report and quarantine the message and follow the organization's incident-response procedure.

Question 2: Identifying a mismatch between Sender and Reply-To

Answer:

The following header fields are particularly useful when investigating a mismatch between the sender and reply destination.

Header field	Purpose
From:	Displays the sender identity presented to the recipient.
Sender:	Identifies the agent responsible for transmitting the message when specified.
Reply-To:	Specifies the address to which replies should be directed.
Return-Path:	Indicates the envelope return address recorded during delivery.
Received:	Records the mail servers through which the message passed.
Authentication-Results:	Records SPF, DKIM, and DMARC authentication results.

Example of suspicious headers

From: IT Support <support@company.example>

Sender: mailer@unknown.example

Reply-To: password-reset@attacker.example

Return-Path: bounce@unknown.example

In this example, the displayed sender claims to represent the company's IT department, while replies are directed to a different domain.

The mismatch may indicate impersonation or an attempt to redirect the employee's response to an attacker. However, a different Reply-To address is not conclusive proof of phishing because legitimate mailing systems and support services may use separate reply addresses.

Investigation procedure

- Compare the domains in the From, Sender, and Reply-To fields.
- Inspect the Return-Path and the sequence of Received headers.
- Review the SPF, DKIM, and DMARC results.
- Verify whether the differing domains belong to an authorized third-party service.
- Correlate the findings with the message content, URLs, and the organization's legitimate email configuration.

Conclusion: A mismatch between the visible sender and Reply-To address is a useful phishing indicator. It should be investigated alongside other evidence before the email is classified as malicious.

Question 3: Display name spoofing despite successful SPF/DKIM checks

Answer:

Display name spoofing is a technique in which an attacker uses a familiar person's or organization's name in the visible sender name while sending the email from a different email address.

For example:

- From: IT Security Team <attacker@external.example>

The recipient may see “IT Security Team” in their email client and assume that the message came from the company's legitimate security department.

SPF and DKIM authenticate aspects of the sending infrastructure and domain. They do not establish that the display name is truthful or that the person using the account is authorized to make the request.

An attacker may therefore create a message that passes authentication checks for a domain they control while using a misleading display name. Alternatively, a legitimate corporate account may be compromised and used to send a malicious email.

Why users can still be deceived

1. Email clients may display the sender's name more prominently than the complete email address.
2. Employees may recognize the name but overlook the actual domain.

3. A message may use familiar company branding, logos, and terminology.
4. Urgent requests may cause employees to act before verifying the sender.
5. Successful authentication can create a false sense of security if users treat it as proof that the content is trustworthy.

Recommended controls

- Display the full sender address for suspicious messages.
- Train employees to inspect the sender's domain.
- Configure DMARC with an appropriate enforcement policy.
- Monitor for impersonation of executives, IT staff, and finance personnel.
- Provide a simple process for reporting suspicious emails.
- Require independent verification of sensitive requests.

Conclusion: SPF and DKIM help establish domain-related authenticity, but they do not validate the truthfulness of the display name or the safety of the email's content.

Question 4: The strategic role of phishing-awareness training

Answer:

Employee phishing-awareness training acts as a human firewall by helping employees recognize suspicious messages and make safer decisions when interacting with email, links, attachments, and authentication prompts.

Technical email filters can identify spam, malicious attachments, suspicious URLs, and known phishing indicators. However, attackers may use newly registered domains, compromised legitimate accounts, social engineering, or messages that have not yet been classified as malicious.

Awareness training complements these technical controls by preparing employees to recognize threats that pass through the filtering system.

Role of awareness training

1. Recognition of phishing indicators: Employees learn to identify suspicious sender addresses, unexpected attachments, urgent requests, and misleading links.
2. Safer authentication practices: Employees learn not to disclose passwords, MFA codes, or session tokens in response to unsolicited requests.
3. Independent verification: Employees learn to verify payment changes, password resets, and sensitive requests through trusted channels.
4. Early incident reporting: Employees are encouraged to report suspicious emails promptly, allowing security teams to investigate and respond.

5. Reduction of human error: Repeated training helps employees develop safer habits when handling unexpected communications.

Combined defense model

Technical email filtering	Employee awareness
Detects known malicious patterns	Recognizes suspicious context
Blocks or quarantines dangerous messages	Avoids unsafe clicks and disclosures
Checks attachments, links, and sender reputation	Verifies unusual requests independently
Provides technical alerts	Reports messages that evade filters

Conclusion: Email filtering and awareness training are complementary controls. Filtering reduces the number of threats reaching employees, while trained employees provide an additional layer of detection and prevention.

Question 5: Immediate response after credentials are compromised

Answer:

I would initiate the organization's incident-response process immediately, prioritizing containment, credential revocation, and investigation.

Step 1: Record and report the incident

- Record the affected user's identity, the time of the incident, and the suspicious email or URL.
- Notify the security operations team and incident-response lead.
- Preserve relevant email, endpoint, identity-provider, and proxy logs.

Step 2: Isolate the affected host

I would isolate the affected endpoint using the organization's endpoint detection and response (EDR) platform or approved network-isolation procedure.

Isolation should prevent communication with attacker-controlled infrastructure while preserving access for authorized incident responders. I would avoid immediately shutting down the system unless the incident-response procedure requires it, because doing so may destroy useful volatile evidence.

Step 3: Revoke compromised credentials and sessions

Using the identity provider's administrative controls, I would:

1. Reset the employee's password using a trusted administrative process.

2. Revoke active sessions and refresh tokens.
3. Revoke or invalidate relevant authentication tokens where supported.
4. Require fresh authentication.
5. Review and remove unauthorized MFA methods, recovery options, or registered devices.
6. Check for unauthorized OAuth grants, application passwords, or persistence mechanisms.

A password reset alone may not terminate an attacker's existing session.

Step 4: Investigate account activity

I would review sign-in logs, IP addresses, device information, MFA events, mailbox rules, OAuth consent grants, and access to sensitive applications.

I would determine whether the attacker accessed other systems or downloaded company data.

Step 5: Remove the threat and recover

I would inspect the affected endpoint for malicious downloads, browser changes, malware, and persistence. After remediation and validation, I would restore the endpoint to the network under the organization's recovery procedure.

Step 6: Prevent recurrence

I would block confirmed malicious indicators, search for similar emails, notify affected users, and document the incident and corrective actions.

Conclusion: The immediate priorities are host containment, credential and session revocation, evidence preservation, and investigation of potential lateral movement or data exposure.

Question 6: NIST CSF category for organizational training

Answer:

The relevant category is Awareness and Training (PR.AT) under the Protect function of the NIST Cybersecurity Framework (CSF).

This category addresses the need for personnel to receive cybersecurity awareness and training so that they can perform their security-related responsibilities and recognize risks such as phishing and social engineering.

Conclusion: NIST CSF Awareness and Training (PR.AT) supports organizational training to mitigate social-engineering risks.

Section B — Practical Coding Tasks

Task 1 Email Authentication Check

Objective

Retrieve a domain's SPF and DMARC DNS records, identify its DMARC policy, and determine whether it publishes p=reject.

Tools required

- Kali Linux
- Linux terminal
- dig or nslookup

Step 1: Install the DNS utilities

On Kali Linux, run:

- `sudo apt update`
- `sudo apt install dnsutils -y`

Verify that dig is available:

- `dig -v`

Step 2: Retrieve the SPF TXT record

For this demonstration, use example.com as a placeholder domain. Replace it with the domain assigned by your trainer.

- `dig TXT example.com +short`

Look for a TXT record beginning with:

- `v=spf1`

A domain may publish multiple TXT records, but it should not publish multiple independent SPF policies for the same DNS name.

Step 3: Retrieve the DMARC record

- `dig TXT _dmarc.example.com +short`

A DMARC record is published at the `_dmarc` subdomain.

An illustrative result might look like:

- `"v=DMARC1; p=reject; rua=mailto:dmarc@example.com"`

This is an example of the record format, not a claim about the current configuration of example.com.

Step 4: Identify the DMARC policy

The three main DMARC policy values are:

Policy	Meaning
p=none	Requests monitoring without a DMARC-directed quarantine or rejection policy.
p=quarantine	Requests that failing messages be treated as suspicious, commonly sent to spam or quarantine.
p=reject	Requests rejection of messages that fail DMARC evaluation.

Step 5: Record the findings

Use this table in your report and fill it using your actual DNS output.

Field	Result
Domain tested	example.com (replace with assigned domain)
SPF record found	Pending actual lookup
DMARC record found	Pending actual lookup
DMARC policy	Pending actual lookup
Is p=reject configured?	Pending actual lookup
Interpretation	Based on the retrieved record

Step 6: Explain the result

If the domain publishes p=reject, it requests that receiving mail systems reject messages that fail DMARC. This helps reduce direct domain spoofing.

However, enforcement depends on the receiving mail system's implementation, and DMARC does not prevent every form of phishing, including messages sent from compromised legitimate accounts or lookalike domains.

Evidence to attach: Screenshot showing the dig commands and their actual output.

Task 2 Phishing Email Header Analysis

Objective

Analyze a raw .eml file, identify its originating IP address, check the IP against a reputation source, and examine Sender/Reply-To mismatches.

Step 1: Locate the email file

Place the .eml file provided by your trainer in your working directory.

- `cd ~/Documents`
- `ls -l *.eml`

If the file is stored elsewhere, change to that directory.

Step 2: Display the raw headers

- `less suspicious_email.eml`

The header block appears before the first blank line separating headers from the message body.

To print the first 50 lines:

- `head -n 50 suspicious_email.eml`

Step 3: Extract the relevant fields

- `grep -iE '^(From|Sender|Reply-To|Return-Path|Received|Authentication-Results):' suspicious_email.eml`

Inspect the complete Received chain. Each receiving mail server may prepend a Received header, so the earliest trustworthy external hop can help identify the originating IP.

Do not automatically trust the first IP-like value in the file: attackers can forge some header fields, and private or internal addresses may appear in the chain.

Step 4: Example header analysis

The following is a fictional example for demonstrating the analysis:

```
From: IT Support <support@company.example>  
Sender: mailer@unknown.example  
Reply-To: login@attacker.example
```

Received: from mail.attacker.example
(203.0.113.25) by mx.company.example
Authentication-Results: mx.company.example;
spf=fail; dkim=none; dmarc=fail

The address 203.0.113.25 belongs to a documentation-only IP range. It is included here purely as an example and must not be treated as a real attacker IP.

Step 5: Check IP reputation

Use a reputation service such as MXToolbox or another approved source provided by your trainer.

For example, open the MXToolbox blacklist lookup page:

Enter the actual IP extracted from the supplied email and record:

- IP address checked
- Date and time of lookup
- Reputation service used
- Whether a listing was reported
- Any relevant listing details

A blacklist result is only one piece of evidence. A clean result does not prove that the IP is safe, and a listing does not independently prove that the particular email is malicious.

Step 6: Document the findings

Indicator	Finding
.eml file	Enter supplied filename
Display from	Extract from the file
Sender	Extract from the file
Reply-to	Extract from the file
Origination IP	Identify from trusted Received chain
SPF result	Extract from authentication results
DKIM result	Extract from authentication results

DMARC result	Extract from authentication results
IP reputaion	Record actual lookup
Assessment	Explain the combined evidence

Task 3 Proxy Log Exfiltration Check

Objective

Filter proxy logs for HTTP POST requests, identify suspicious external destinations, and correlate them with the indicators found in Task 2.

Step 1: Locate the proxy log

Assume your trainer has provided a file named proxy.log.

- `cd ~/Documents`
- `ls -l proxy.log`

View the first few entries:

- `head -n 20 proxy.log`

First, identify the log format. The exact parsing command depends on whether the file uses a standard proxy format, JSON, or custom fields.

Step 2: Filter HTTP POST requests

For a log where the HTTP method appears as a standalone field, use:

- `grep -E '"POST | POST ' proxy.log`

This is a basic filter. Review the output to ensure it matches the actual format of your supplied log.

If the log uses JSON, use a JSON-aware parser rather than relying on text matching.

Step 3: Identify external destinations

For each POST request, record:

- Timestamp
- Source IP address
- Destination domain or URL
- HTTP status, if available

- Whether the destination is internal, recognized, or external
- Whether the destination matches an indicator from Task 2

An unfamiliar domain is not automatically malicious. Check whether it is an approved business service and whether the request is consistent with normal application activity.

Step 4: Example proxy-log entries

The following entries are fictional examples, not findings from your actual log.

2026-09-28T10:15:21 192.168.1.25 POST https://login.company.example/session

2026-09-28T10:17:42 192.168.1.25 POST https://attacker.example/collect

2026-09-28T10:18:03 192.168.1.30 GET https://updates.company.example/

The second entry is suspicious in this fictional scenario because it is an external POST to a domain associated with the investigation. It requires further validation.

Step 5: Create the findings table

Use this format for the actual flagged requests.

Timestamp	Source IP	Destination domain	Method	Reason flagged
Actual log value	Actual log value	Actual domain	POST	Correlates with Task 2 indicator
Actual log value	Actual log value	Actual domain	POST	Unrecognized external destination

If no matching requests are found, state that no matching requests were identified in the supplied log. Do not create findings simply to populate the table.

Step 6: Correlate the evidence

Compare the suspicious IP or domain from Task 2 with the destination domains in the proxy log.

If a POST request matches the suspicious indicator, investigate the timestamp, originating endpoint, surrounding requests, and associated identity-provider events. A POST request alone does not prove credential exfiltration; the destination and request context must support that conclusion.

Conclusion: Proxy-log analysis can help identify possible credential submission or data exfiltration. Correlation with email headers, reputation information, and authentication logs increases confidence in the investigation.

Evidence to attach: Screenshot of the filtered POST requests and the completed findings table.

Task 4 Attack Simulation and Mail Flow Defense

Objective

Demonstrate how a simulated phishing page can be used to understand credential-harvesting risks and how an external-email warning banner can help users identify suspicious messages.

Part A: Controlled phishing-awareness simulation

The assessment specifies the Social Engineering Toolkit (SET) and a simulated corporate login page. This exercise should be performed only in an isolated lab with explicit authorization.

For this submission, use a non-credential-collecting awareness simulation. The page should not collect real passwords, MFA codes, session cookies, or other authentication secrets.

Step 1: Prepare the lab

1. Use an isolated virtual machine or private lab network.
2. Use a fictional company identity and a dummy login page.
3. Do not clone a real third-party login portal.
4. Ensure the simulation cannot be accessed by unintended users.
5. Use a page that displays a training message rather than recording submitted credentials.

Step 2: Demonstrate the phishing indicators

Create a mock login page with the following visible training indicators:

- A fictional organization name
- A warning that the page is part of a security-awareness exercise
- A non-functional demonstration login form
- A message explaining that real credentials must never be entered into untrusted pages

The purpose is to demonstrate how branding and familiar login layouts can influence user trust, without creating a working credential harvester.

Step 3: Capture evidence

Capture screenshots showing:

1. The simulated awareness page.

2. The lab environment and test setup.
3. The training message displayed after the demonstration interaction.

If your instructor specifically requires a SET-generated credential-harvesting page, coordinate with them to use an approved, isolated, non-collecting training configuration.

Part B: Configure an external-email warning banner

A mail-flow rule can prepend a warning to messages received from outside the organization.

The exact menu names vary by mail platform. The following describes the configuration requirements for a simulated Microsoft Exchange mail environment.

Step 1: Define the rule

Use a rule name such as:

- External Email Warning Banner

Configure the rule to identify incoming messages originating outside the organization's trusted internal domains.

Step 2: Configure the action

Prepend a clearly visible banner to matching messages.

Example banner:

WARNING: EXTERNAL EMAIL

This message originated outside the organization. Verify the sender and inspect links carefully. Never disclose your password, MFA code, or session token in response to an unexpected request.

Step 3: Test the rule

Send a test message from an external test account to a lab mailbox.

Verify that:

- The warning banner is visible.
- The message content remains readable.
- Messages from trusted internal senders are handled according to the rule's intended scope.
- The banner does not interfere with normal mail delivery.

Step 4: Record the configuration

Configuration item	Value
Rule name	External Email Warning Banner
Condition	Incoming message from outside the organization
Action	Prepend external-email warning
Test performed	Record actual test
Result	Record actual result

Part C: Explain how the control reduces risk

An external-email banner provides a visible reminder that a message originated outside the organization. It may encourage employees to check the sender's address, inspect links, and independently verify sensitive requests.

In the scenario from Tasks 2 and 3, the banner could have prompted the recipient to investigate an unfamiliar sender or suspicious destination before submitting credentials.

However, it is not a complete defense. Attackers may use compromised internal accounts, and users may become accustomed to warning banners. The control should be combined with email authentication, filtering, awareness training, phishing-resistant MFA, and monitoring.

Conclusion: A non-collecting phishing-awareness simulation demonstrates the human risk, while an external-email banner provides an additional visual control. The effectiveness of both controls should be tested and documented.

Evidence to attach: Screenshot of the mock awareness page, the mail-flow rule configuration, and a received test email displaying the warning banner.

Section C — Mini Capstone Project

Apex Financial: Spear-Phishing and Session Hijack Incident Response

1. Executive Summary

Apex Financial experienced a confirmed adversary-in-the-middle (AitM) spear-phishing incident. An employee's password and MFA session token were captured through a cloned login portal. The attacker subsequently used the stolen session to access the HR database and exfiltrate personally identifiable information (PII).

This incident demonstrates that a password and MFA challenge can both be defeated when an attacker captures and reuses an authenticated session.

The response must prioritize termination of the attacker's existing access, containment of the affected account and endpoints, preservation of evidence, investigation of the HR database access, and prevention of similar incidents.

The principal response actions are:

1. Revoke active sessions and refresh tokens.
2. Reset the affected user's password and review authentication factors.
3. Contain compromised endpoints and block confirmed malicious infrastructure.
4. Investigate identity-provider, endpoint, and HR database logs.
5. Determine the scope of PII exposure and preserve evidence.
6. Deploy phishing-resistant authentication and device-compliance controls.

2. Incident Overview

Item	Description
Organization	Apex Financial
Incident type	Spear-phishing and session hijacking
Attack technique	Adversary-in-the-middle phishing
Initial access	Cloned login portal
Credentials compromised	Employee password
Additional compromise	MFA session token
Targeted resource	HR database
Impact	Unauthorized access and PII exfiltration
Status	Confirmed incident; investigation and remediation required

3. Technical Indicators of Session-Token Theft

A stolen password and a stolen session token are different forms of compromise. A password is an authentication secret, while a session token can represent an already-authenticated session.

The following indicators would support the conclusion that a session token was stolen and reused.

3.1 Successful authentication followed by suspicious session activity

Identity-provider logs may show that the employee successfully completed authentication, including MFA, followed by unexpected access from a different IP address, device, or location.

A change in IP address alone is not conclusive because legitimate users may change networks. The event must be correlated with other indicators.

3.2 Reuse of an existing authenticated session

The attacker may access applications using an existing session without generating a new password or MFA challenge.

Evidence may include:

- A session identifier or token family associated with both legitimate and suspicious activity.
- Continued access after the employee has stopped interacting with the service.
- Application access without a corresponding new interactive authentication event.
- Suspicious reuse of refresh tokens, where the identity provider records it.

Raw session tokens should not be copied into the report. Record appropriate event identifiers and securely preserved evidence instead.

3.3 Unusual device and network characteristics

The session may be used from an unfamiliar device, IP address, user agent, or network. These characteristics become more significant when they coincide with suspicious authentication or access events.

3.4 HR database access anomalies

Database audit logs may show queries, exports, or downloads outside the employee's normal activity. Correlate the database events with the identity-provider session and the timestamps of the phishing incident.

3.5 Evidence of adversary-in-the-middle activity

Additional indicators may include a phishing URL resembling the genuine identity provider, a reverse-proxy domain, and a sequence of authentication events consistent with a relayed login.

Conclusion: The strongest confirmation comes from correlating identity-provider session records, token-reuse events, endpoint evidence, and HR database audit logs. No single IP-address change is sufficient on its own.

4. Containment and Session Revocation

The following sequence should be executed by authorized identity, endpoint, and security administrators.

Step 1: Declare and coordinate the incident

- Open an incident record.
- Identify the incident commander and responsible technical teams.
- Record the known timeline, affected employee, and affected systems.
- Preserve relevant logs and evidence before routine retention processes remove them.

Step 2: Disable or restrict the affected account

Temporarily block sign-in for the affected account if necessary to prevent further access while the investigation proceeds.

Coordinate this action with the employee's manager and business owners to limit unnecessary disruption.

Step 3: Revoke all active refresh tokens and sessions

Use the identity provider's administrative session-revocation functionality.

The response must include:

- Revocation of refresh tokens for the affected user.
- Termination or invalidation of active sign-in sessions.
- Revocation of application sessions where the application maintains its own session state.
- Invalidation of relevant tokens through the provider's supported mechanisms.
- Review of the maximum session lifetime and token-revocation behavior.

Section D — AI-Augmented Learning

Step 1: Build with AI

Objective

Write a Bash script that reads raw email headers, extracts the Sender and Reply-To addresses, compares their domains, and reports mismatches with an originating IP address.

1.1 Exact prompt given to the AI tool

The following is the prompt used to generate the initial script:

Write a Bash script for Kali Linux that reads a text file containing raw email headers, with one email per block. Extract the Sender and Reply-To email addresses from each block, compare their domains without considering uppercase or lowercase differences, and flag messages where the domains do not match. Print a summary report containing the email number, Sender address, Reply-To address, and originating IP address extracted from the Received headers. Include basic error handling and comments explaining the code.

1.2 AI's original code

Save the following as email_check_original.sh.

```
#!/bin/bash

# Basic email header mismatch checker

FILE="$1"

if [ ! -f "$FILE" ]; then
    echo "Usage: $0 <header_file>"
    exit 1
fi

echo "Email Header Analysis Report"
echo "-----"

awk 'BEGIN { RS=""; FS="\n" }
{
    from=""
    reply=""
    ip=""

    for (i=1; i<=NF; i++) {
        if ($i ~ /^From:/) {
            from=$i
        }
    }
}
```

```

    if ($i ~ /^Reply-To:/) {
        reply=$i
    }

    if ($i ~ /^Received:/ && ip=="") {
        if (match($i, /[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+/)) {
            ip=substr($i, RSTART, RLENGTH)
        }
    }
}

if (from != "" && reply != "") {
    if (tolower(from) != tolower(reply)) {
        print "Sender: " from
        print "Reply-To: " reply
        print "Originating IP: " ip
        print "Status: FLAGGED"
        print ""
    }
}
}' "$FILE"

```

1.3 Explanation of the original code

- FILE="\$1" reads the input filename supplied to the script.
- The file check verifies that the input exists.
- RS="" tells awk to process blank-line-separated blocks as records.
- The loop searches for From, Reply-To, and Received headers.
- The regular expression extracts an IPv4-looking string from a Received header.
- The script flags a message when the complete From and Reply-To header lines differ, ignoring letter case.

1.4 Limitation identified

The original script has a logical bug: it compares the complete From and Reply-To header lines instead of comparing only their email domains.

For example:

```

From: IT Support <support@company.example>
Reply-To: Help Desk <help@company.example>

```

The names and email addresses differ, but both email addresses belong to company.example. The original script would incorrectly flag this message.

It also selects the first matching Received header and may therefore report an intermediate mail server rather than the originating IP.

Step 2: Test and Debug Without AI

2.1 Create a sample input file

Create a file named sample_headers.txt:

```
From: IT Support <support@company.example>
Reply-To: Help Desk <help@company.example>
Received: from relay.company.example (192.0.2.10)
Received: from sender.example (198.51.100.25)

From: Security Team <security@company.example>
Reply-To: Login <attacker.example@evil.example>
Received: from suspicious.example (203.0.113.25)
```

The IP addresses are reserved documentation examples, not real attacker addresses.

2.2 Run the original script

- `chmod +x email_check_original.sh`
- `./email_check_original.sh sample_headers.txt`

Expected observation: The original script flags the first email even though the Sender and Reply-To domains are both company.example. This demonstrates the domain-comparison bug.

Record the actual output from your terminal in your submission.

2.3 Corrected Bash script

Save this version as email_check_corrected.sh.

It compares the extracted email domains, handles common folded headers, and selects the earliest Received entry in the header chain as a candidate originating IP. That IP remains a candidate because the header chain may contain untrusted entries.

```
#!/usr/bin/env bash
```

```
# Email Sender/Reply-To domain mismatch checker
```

```

# Usage: ./email_check_corrected.sh <header_file>

set -u

FILE="${1:-}"

if [[ -z "$FILE" || ! -f "$FILE" ]]; then
    echo "Usage: $0 <header_file>" >&2
    exit 1
fi

# Extract an email address from a header value.
extract_email() {
    local value="$1"

    if [[ "$value" =~ \<([^\<[:space:]]+@[^\<[:space:]]+)\> ]]; then
        printf '%s' "${BASH_REMATCH[1]}"
    elif [[ "$value" =~ ([^\[:space:],\<\>]+@[^\[:space:],\<\>]+) ]]; then
        printf '%s' "${BASH_REMATCH[1]}"
    fi
}

# Return the domain portion of an email address.
extract_domain() {
    local address="$1"
    local domain

    [[ "$address" == *@* ]] || return 1

    domain="${address##*@}"
    domain="${domain%}"
    domain="${domain,,}"

    # Reject empty or malformed domains.
    [[ -n "$domain" && "$domain" != "*"@* ]] || return 1

    printf '%s' "$domain"
}

# Process one normalized header block.

```

```

process_block() {
    local block="$1"
    local from="" reply="" received=""
    local address_from="" address_reply=""
    local domain_from="" domain_reply=""
    local candidate_ip=""

    # Unfold continuation lines and extract relevant headers.
    while IFS= read -r line; do
        case "$line" in
            [Ff][Rr][Oo][Mm]:*)
                from="$ {line#*:} "
                ;;
            [Rr][Ee][Pp][Ll][Yy]-[Tt][Oo]:*)
                reply="$ {line#*:} "
                ;;
            [Rr][Ee][Cc][Ee][Ii][Vv][Ee][Dd]:*)
                # Received fields appear newest-first.
                # Retain the last one encountered.
                received="$line"
                ;;
        esac
    done <<< "$block"

    address_from="$(extract_email "$from")"
    address_reply="$(extract_email "$reply")"

    if [[ -z "$address_from" || -z "$address_reply" ]]; then
        return
    fi

    domain_from="$(extract_domain "$address_from")" || return
    domain_reply="$(extract_domain "$address_reply")" || return

    if [[ "$domain_from" != "$domain_reply" ]]; then

        # Extract a candidate IPv4 address from the
        # oldest Received header in the block.
        if [[ "$received" =~ ([0-9]{1,3}\.){3}[0-9]{1,3} ]]; then
            candidate_ip="${BASH_REMATCH[0]}"
        fi
    fi
}

```

```

else
    candidate_ip="Not found"
fi

((flagged++))

printf 'Email: %d\n' "$email_number"
printf 'Sender: %s\n' "$address_from"
printf 'Sender domain: %s\n' "$domain_from"
printf 'Reply-To: %s\n' "$address_reply"
printf 'Reply-To domain: %s\n' "$domain_reply"
printf 'Candidate originating IP: %s\n' "$candidate_ip"
printf 'Status: FLAGGED - DOMAIN MISMATCH\n\n'
fi
}

# Normalize folded headers and process blank-line blocks.
email_number=0
flagged=0
block=""

while IFS= read -r line || [[ -n "$line" ]]; do

    # A continuation line begins with whitespace.
    if [[ "$line" =~ ^[:space:]]+ && -n "$block" ]]; then
        block+=$'\n'"$line"
        continue
    fi

    if [[ -z "$line" ]]; then
        if [[ -n "$block" ]]; then
            ((email_number++))
            process_block "$block"
            block=""
        fi
    else
        block+="${block:+$'\n'}$line"
    fi
fi

done < "$FILE"

```

```
# Process the final block if the file has no trailing blank line.
if [[ -n "$block" ]]; then
    ((email_number++))
    process_block "$block"
fi
```

```
echo "-----"
echo "Emails processed: $email_number"
echo "Flagged emails: $flagged"
```

2.4 Run the corrected script

- `chmod +x email_check_corrected.sh`
- `./email_check_corrected.sh sample_headers.txt\`

For the sample input, the expected findings are:

Email	Sender domain	Reply-To domain	Expected result
1	company.example	company.example	Not flagged
2	company.example	evil.example	Flagged

The second message should be reported as a domain mismatch. The candidate IP printed by the script is based on the sample's oldest Received header.

2.5 Further limitations

The corrected script is still a lightweight header-analysis tool, not a complete email-forensics parser.

- It does not validate every part of an email address according to the full email-address standard.
- The oldest Received header is not necessarily trustworthy; an analyst must identify the trusted mail-server boundary.
- The script extracts an IPv4-looking string but does not validate that each octet is within the valid range.
- It does not parse every possible MIME or malformed-header edge case.
- A domain mismatch is an investigation indicator, not proof of phishing.

2.6 Required 3–4 line note

Bug identified: The original AI-generated script compared the entire From and Reply-To header lines, causing false positives when the display names or email addresses differed but the domains matched.

Correction: I modified the script to extract the email addresses and compare their domains case-insensitively. I also added handling for common folded headers and improved the originating-IP selection.

Reason: These changes make the comparison more relevant to the assignment and reduce false positives. The script still requires manual validation of the Received chain and other email-forensics evidence.